Real IBM quantum computer | Real data | Honest limits

Quantum Weather Report

The complete guide, from zero to finished: what it is, how it works, what we did, what we found, and what happens next. Written in simple words, with real-life examples.

Project: github.com/Knsravan/qpu-weather | Live dashboard: knsravan.github.io/qpu-weather | Explainer page: knsravan.github.io/qpu-weather/explain.html

Numbers in this guide come from the two real runs on IBM's **ibm_fez** chip on 2026-10-09. Where something is an estimate or unproven, this guide says so.

1. The whole project in one page

The problem. A quantum computer makes mistakes, and how many it makes changes from day to day. The company that owns it (IBM) publishes a table of "how accurate each part is". That table is IBM's own measurement, and it can be out of date.

The idea. Be an independent inspector. Every week, run small tests on a real IBM quantum computer, measure the mistakes ourselves, and compare with IBM's table. Then pick the best parts of the chip, and find out which "error-fixing trick" gets the most correct answers.

The real-life version. A school bus company says "we're usually on time". You ride the bus every week and write down what really happens. Over time you can say "take the 8:10 bus on Mondays, leave early on Fridays."

What we have built

A program that does all of this automatically every Monday on GitHub, saves the results, and shows them on a public web dashboard. It has already run twice on a real IBM chip and produced real results.

The honest summary

This is a small, honest, working project, not new science. Its strength is real hardware, real data and clear limits. Nothing in it should be called "proven" or "certified".

2. The characters (real thing = school story)

Imagine a **school with 156 students**. We will use this picture for the whole guide.

Real thing	In our school story	What it really is
Quantum computer (QPU)	The school	A very sensitive chip. IBM owns it; we rent a little time on it for free.
Qubit	One student holding up a card showing 0 or 1	The quantum version of a bit. The chip we used, *ibm_fez*, has 156.
Readout	A scribe standing next to each student, writing down what the card shows	The last step of every program: reading the qubit's value.
Readout error	The scribe misreads the card	Student showed 0, scribe wrote 1 (or the reverse). The student did nothing wrong.
Gate	An instruction to a student, e.g. "flip your card"	One basic operation on a qubit.
Connection (two-qubit gate)	A handshake between two neighbouring students	The way two qubits work together. The chip has 176 neighbouring pairs.
Shot	One try	Each program is repeated thousands of times and the outcomes are counted.
IBM's table (datasheet)	The principal's report card	IBM's own published accuracy numbers for each part.
This project	The independent inspector	Tests every student, scribe and handshake itself.
Drift	Students having good and bad days	The chip's behaviour changes over hours and days.
Error mitigation	Tricks to get a better answer despite mistakes	Software methods that reduce the effect of noise.

3. How this project came about

1. **First idea: a "certified quantum random number generator".** Use the chip to make random numbers and test them. You decided this felt niche and not useful in real life. (That was a fair point: normal computers already make random numbers that are good enough for almost everyone.)

2. **New requirement:** a useful project on a **real quantum computer**, not a simulator, with real outputs.
3. **Chosen:** a "hardware health tracker" with two fixes for real gaps: **measure the chip yourself** instead of trusting the vendor, and **measure which error-fixing trick really works** instead of using a default.
4. **Dropped:** comparing different companies' chips. It is too costly and hard for one developer.

4. What happens in one run, step by step

1	2	3	4	5	6	7
Check the scribes	Check stamina	Check handshakes	Pick 5 best	Quiz	Try 4 fixes	Save + show

Step 1: Are the scribes accurate? (readout error)

The inspector knows which card the student is holding, because the inspector told them. So the inspector can check the scribe.

1. Tell the student "hold up **0**". Repeat 2,000 times. Count how often the scribe wrote 1 by mistake.
2. Tell the student "hold up **1**". Repeat 2,000 times. Count how often the scribe wrote 0 by mistake.
3. Average the two mistake rates. That is the qubit's **readout error**.

Example only: if the scribe got 20 out of 2,000 wrong, that is 1%.

> **What we found**
> Over all 156 qubits, our typical (median) number divided by IBM's number was **1.01**. So the principal's report card is accurate for the typical student. But a few scribes were much worse than the report card said. For example **qubit 90**: we measured **5.83%**, IBM said **0.43%**.

Step 2: Stamina test (single-qubit gate sequence)

Tell a student "flip your card 100 times". Flipping an even number of times should return the card to where it started. If it is not back, the student slipped somewhere. This is like asking someone for 100 push-ups and counting the wobbles.

> **Important caution**
> This number came out about **13 times higher** than IBM's single-gate figure and was unrelated to it. The test also catches the student getting tired (the qubit simply losing its state over time) and neighbours distracting them. So we do **not** claim this is the true gate error. We only use it to compare qubits with each other.

Step 3: Handshake test (two-qubit connections)

Some handshakes undo themselves when you do them twice, like a **light switch**: flip it twice and it is back as it started. So:

1. Start with two neighbouring students both showing 0.
2. Make them do the handshake **20 times**. An even number, so ideally everything ends where it started.
3. Check how often both are still showing 0. If one went wrong, a handshake was imperfect.

Real numbers: the typical connection had an error of about **0.85% per handshake**. After 20 handshakes, about 84% of the time both students are still right. We work out the per-handshake error from that.

We test each pair **two ways**, starting from two different setups, because some mistakes cannot be seen from the first setup alone. In our real run, the first setup almost never caught anything on this chip's handshake (typical 0.01%); the second setup caught the real signal (typical 0.85%). Without the second test, this step would have looked falsely perfect. We also correct for the scribes' misreading first, so scribe mistakes do not hide real handshake mistakes.

> **How to read it**
>
> Our handshake numbers were about **2.6 times higher** than IBM's, and the two lists agree only partly on which connections are good or bad. The headline correlation of 0.73 comes mostly from four connections that IBM lists at 100% error; the rank correlation is only about 0.4. Treat it as a **rough indicator**, not an exact error rate and not a validated ranking. It also includes some tiredness and neighbour interference, and it is not the professional method (randomized benchmarking).

Step 4: Pick the relay team of 5

A small program needs a row of 5 connected qubits. Like a relay race, **each runner** must be good and **each baton handover** must be clean. The tool tries every possible row of 5 connected qubits and gives each a "cost":

`cost = for each of the 5 qubits (readout error + 3 x stamina error) + for each of the 4 connections (handshake error)`

Lowest cost wins. The comparison is the careless choice: just take the first 5 qubits (numbers 0 to 4).

Real result (latest run): our pick, qubits 94, 95, 99, 115, 114, scored **0.128**, against **0.288** for the careless row. Lower is better, so about twice as good. Note that the earlier run picked a different row, qubits 24, 25, 37, 45, 46. The "best row" is a recommendation for that moment, not a permanent fact.

Step 5: The quiz (benchmarks with a known answer)

To see how well the team really performs, we give them quizzes where we already know the answer key:

- **Mirror quiz:** walk forward through some random steps, then walk the exact same steps back. You must end where you started, which is the answer "all zeros".
- **GHZ quiz:** a second quiz whose right answer is either "all zeros" or "all ones".

Each quiz is repeated **4,000 times**. **Score = the share of attempts that gave the right answer**. 1.0 is perfect. The tool runs the quizzes on both the best row and the careless row.

Step 6: Try four "fixes" and see which helps most

Fix	In plain words	Everyday example
raw	Do nothing special. Count the right answers.	Just take the test.
readout fix	We know each scribe's misread rate, so we correct the final tally mathematically.	A thermometer reads 2 degrees too high; you subtract 2.
noise trick (zero-noise extrapolation)	Run the program with *more* noise on purpose (3x and 5x as many steps), then draw a line and guess what the result would be with *zero* noise.	Measure how much a car slows with 1, 3 and 5 passengers, then guess its speed with none.
both	Readout fix first, then the noise trick.	Correct the thermometer, then do the passenger trick.

Careful: the noise trick draws a straight line, so it can overshoot, and we cap results at 1.0. A score of exactly 1.000 is that cap, not a perfect chip.

Step 7: Save and show

The results are saved as a data file in the project, and the web dashboard reads it. A GitHub robot does all of this automatically every **Monday at 03:17 UTC (about 8:47 AM in India)**.

5. The results, in plain words

Method	Average score (best row, both runs)	Mistakes removed vs raw
raw	0.915	-
readout fix	0.951	42%
noise trick	0.948	39%
both (best)	**0.983**	**80%**

How to read "80%": raw scored 0.915, so about 8.5% of tries were wrong. "Both" scored 0.983, so only about 1.7% were wrong. That removes 80% of the mistakes.

Three results we can stand behind

1. **IBM's readout numbers matched our measurement** (1.01 times, typical), though a few qubits were clearly worse than advertised.
2. **Choosing qubits well helped.** On the best row the average raw score was about 0.915; on the careless row about 0.80.
3. **Using both fixes together was best** in every case. A published best-practices paper reports that a similar combined strategy improved results on an IBM device, so this is consistent with known work.

What we have NOT shown

- **Drift over time.** Only 2 runs, both from the same day. The weekly runs will build this.
- **Exact gate error rates.** The gate numbers are quick estimates; their value for ranking is not validated.
- **Anything about other chips or other days.** This is one chip.
- A 2026 research summary (which I could not verify) suggests the extra shots needed for some fixes can cancel their benefit when you have a tight shot budget. Our scores do not test that.

Reading each part of the dashboard

Dashboard part	What it tells you
Today's advice	The best row of 5 qubits on that day, with the cost compared to the careless choice.
Does reality match the datasheet?	The typical ratio of our readout measurement to IBM's. 1.0 means they agree. "Worst offenders" lists qubits that are much worse than IBM says.
Which error fix works best?	The table in section 5, averaged over all recorded runs.
Drift chart	Blue = what we measured, red = what IBM reports, over time. Currently nearly flat, because only two same-day points exist.
Five best / worst qubits	The qubits with the lowest and highest readout error, with the stamina column clearly labelled as *not* comparable to IBM's figure. One qubit (number 72) is listed by IBM with a 100% gate error, which looks like IBM's way of marking it unusable (my reading; not confirmed).

6. What we actually did in this build

The project's code existed before, but it had only ever run on a **simulator** (a pretend quantum computer). The work was to make it run on a **real** one, and to make it honest.

What happened	Details
First attempts failed	IBM replied "API key could not be found". The key had been deleted. After creating a fresh key, login worked.
A run hung for an hour	IBM accepted the job but would not run it, because the free time looked used up. Nothing in our program told us. It was killed after 60 minutes.
Safety guards added	The program now stops early with a clear message if there is no quantum time left, and cancels any job still waiting after 10 minutes.
Schedule changed	From daily to weekly, because free quantum time is limited.
First real run worked	On ibm_fez (156 qubits). Results were saved and the dashboard displayed them. This proved login, choosing a chip, running both jobs, reading results, saving and publishing all work.
Two-qubit measurement added	Instead of trusting IBM's connection numbers, we measure them (the handshake test). Testing on the simulator showed a first version read near zero; we fixed it by correcting for scribe mistakes and adding the second starting setup. Then it ran for real on 176 connections.
Dishonest number relabelled	The stamina number looked 13 times worse than IBM's. Rather than present that as "the chip is 13x worse", we relabelled it and use it only for ranking.
A bug caught before going live	My own dashboard edit had a typing mistake that would have blanked the page. A browser test found it before it reached the live site.
Explainer page and this guide	An interactive page where you point at any dashboard part and get an explanation, plus this document.

Quantum time used: IBM's usage page showed about 41 seconds. It is not confirmed whether that is per run or for both runs together. Either way a weekly run fits comfortably.

Tests: 27 automatic tests check the logic. They never need a quantum computer.

7. Honest limits (say these out loud)

- The readout comparison is solid. The gate numbers are **quick estimates**, used inside the qubit-ranking score; how good that ranking is has not been validated.
- Everything describes **one chip on one day**. Do not generalise.
- Only two runs exist so far, both from the same day. **No drift story yet.**
- Passing the quizzes proves nothing beyond those quizzes.

- The "best row" changed between two runs minutes apart, so it is a per-run recommendation.
- **Our chain was only compared with the first five qubits**, not with the row IBM's own software would pick from IBM's numbers. That comparison has not been run yet, so "our row beats a careless row" does not mean "our row beats IBM's pick".
- Free quantum time is limited and its rules can change.

8. Who it is useful for, and what is new

Useful for

- **Learners** who want to see real hardware numbers next to the datasheet.
- **Free-plan IBM users** choosing which qubits to run on.
- **Anyone curious** whether a vendor's published numbers hold on a given day.
- **Your portfolio:** real hardware, real data, honest limits.

Not useful for: choosing between quantum companies (only one chip), research-grade error rates, or production work.

What is new, honestly

I did three quick web searches on 2026-10-11. That is not a full literature review.

Piece	Exists already?
Re-running benchmarks regularly to track a chip	**Yes.** Metriq (Unitary Foundation) does this for standard benchmarks.
IBM chips drifting day to day	**Yes.** Academic studies show it.
Readout fix + noise trick beats either alone	**Yes.** A published best-practices paper reports composite mitigation improving results on an IBM device. Our result is consistent with it.
A public, automatic page comparing IBM's reported numbers with independent measurement, per qubit, kept as history	**Not found** in one search. I cannot prove nobody else has it.
Four fixes ranked together on the same circuits, every run	Pieces exist (Mitiq, papers); a repeating automatic version was not found.

> **The honest one-line claim**
>
> "An automated, open weekly check of a real IBM chip against its own datasheet, plus a measured ranking of error fixes. Incremental, with real data and honest limits."

Sources: Metriq paper (arxiv.org/html/2603.08680v1); IBM chip drift study (arxiv.org/pdf/2201.02899); digital ZNE best practices (arxiv.org/pdf/2307.05203); IBM, "refresh backend properties with real-time benchmarking" (quantum.cloud.ibm.com/docs/ja/tutorials/refresh-backend-properties-with-real-time-benchmarking).

9. What happens next, and what you do

Automatically

Every Monday the robot runs the checks on a real chip, saves a results file, and updates the dashboard. You do not need to do anything. If IBM free time runs out, the run stops with a clear message.

What you do now

- Nothing is required. The project is finished and pinned on your profile.
- Do not delete the `IBM_QUANTUM_TOKEN` secret. If you ever create a new IBM key, update that secret.

What you do later

1. **Mondays:** open the repo's **Actions** tab and check for a green run. If one is red, open it and read the message.
2. **Now and then:** look at IBM's usage page to see time used.
3. **After about 4 weeks (4 or more runs):** look at the dashboard and write 3 to 5 sentences in the README: Does the drift chart move? Is the best row stable? Does "both fixes" still win? Use only what the data shows. A template: *"Over N weekly runs on ibm_fez, median readout error stayed within X to Y% and matched IBM's table to within Z times. The best row changed in K of N runs, so choosing qubits fresh each time mattered. Readout plus noise trick scored highest in N of N runs."*
4. **Optional upgrades** (from the project notes): the professional gate-measurement method, more error fixes, a stability score per qubit, other chips.

10. Interview cheat sheet

> **30-second answer**
>
> "Vendors publish accuracy numbers for their quantum chips, but those can go stale. I built a tool that runs small tests on a real IBM quantum computer every week, measures the errors itself, compares them with IBM's numbers, picks the best qubits, and tests which error-fixing method gets the most correct answers. It runs automatically on GitHub and shows the results on a public dashboard. On a 156-qubit chip, IBM's readout numbers matched mine, picking qubits well helped, and combining two error fixes removed about 80% of the mistakes."

Why not use a simulator?

A simulator only contains the noise someone put into it. I wanted to see what a real chip does, and where it differs from the datasheet.

What went wrong and how did you fix it?

The first real runs failed on login, then one run hung for an hour because the free quantum time was used up. I added a check that stops early with a clear message, and a limit that cancels jobs stuck in the queue.

Which result do you trust, and which do you not?

I trust the readout comparison and the fix ranking (it is consistent with published work). I do not trust the single-qubit gate number as an absolute error (it came out 13 times too high because it includes the qubit decaying and neighbour interference), so I relabelled it and only use it for ranking. My two-qubit measurement is about 2.6 times IBM's and agrees with IBM's ranking only moderately (rank correlation about 0.4), so it is a rough indicator, not a validated ranking.

What is new?

Honestly, nothing in the science. Benchmark tracking and the fix comparison already exist. The small gap is an open, automatic tracker of "vendor's reported numbers versus independent measurement" per qubit. I could not find one in a quick search, but I cannot prove none exists.

What would you do next?

Collect a few weeks of data to look at drift, replace my quick two-qubit test with randomized benchmarking, and test whether the extra shots needed for the fixes are worth it on a fixed budget.

How do you know your code works?

27 automatic tests (no quantum computer needed), plus two real runs on hardware. I also tested the measurement methods on a simulator where I know the true answers, and found and fixed a flaw that way.

11. Mini glossary

Qubit	One unit of a quantum computer. The chip we used has 156.
Shot	One try of a program. We repeat each 2,000 to 4,000 times.
Readout error	The chip reads a qubit's value wrongly at the end.
Gate	One basic operation. One-qubit gates act on one qubit; two-qubit gates act on a connected pair.
Median	The middle value when you line up all numbers. Hides extremes, so we also show the worst ones.

Drift	The chip's behaviour changing over time.
Benchmark	A test program with a known right answer.
Error mitigation	Software tricks that reduce the effect of noise.
Zero-noise extrapolation	Run with extra noise on purpose, then guess the zero-noise result from the trend.
GitHub Action	The robot on GitHub that runs the project automatically every Monday.
Dashboard	The public web page that shows the results.

End of guide. Everything here is checked against the project files and the two real run files. If anything in the project changes, the project's CLAUDE.md file holds the up-to-date technical brief.